

ALGORITHM AND DATA STRUCTURES
ASSIGNMENT 2 – LINKEDLIST



By:
I Kadek Mahesa Permana Putra
F1D02410052

FACULTY OF ENGINEERING
DEPARTEMENT OF INFORMATICS ENGINEERING
UNIVERSITY OF MATARAM
2025

A. DIRECTORY

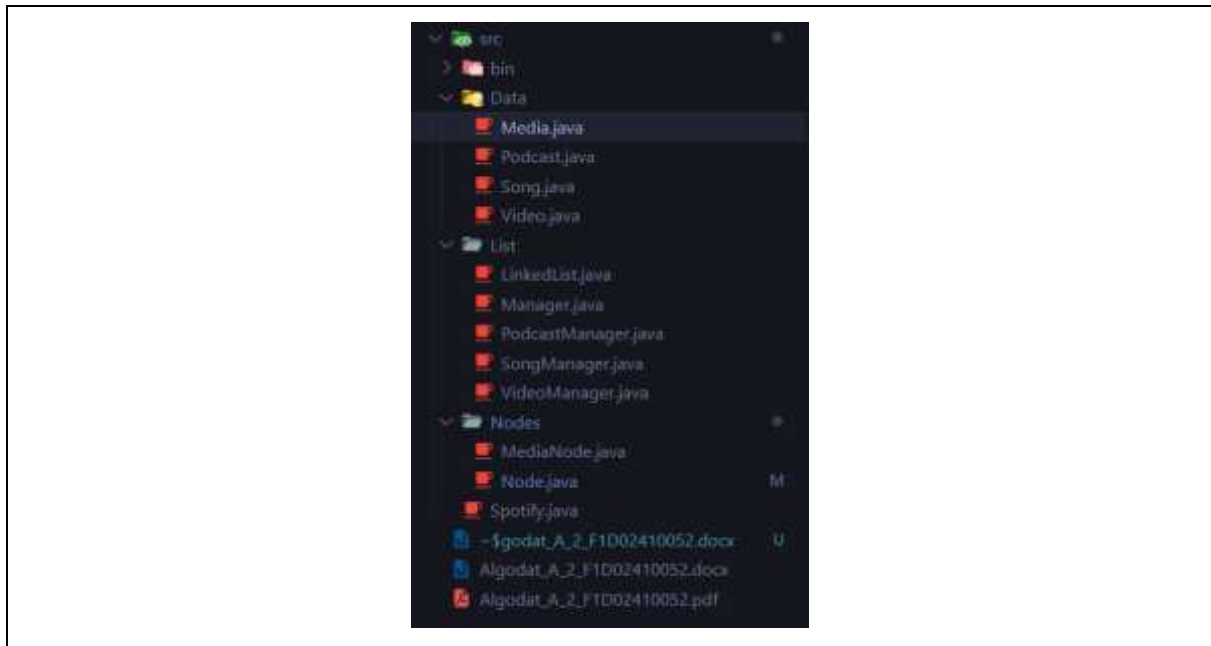


Image 1. Project Directory

This is the Project's Directory, this image show the hierarchy of the files used for this program, the **Main** folder contains the main program which is **Spotify.java**, and then the other directory such as **src/Data** used to store the abstract data type used for this program. **src/List** for the LinkedList library and lastly **src/Nodes** stores the different types of Nodes in this case **Abstract Node** and the implementation in **MediaNode.java**

B. SOURCE CODE

NODES

```
package Nodes;
import Data.Media;

public class Node {
    public Media data;
    public Node nestedNode;
    public Node next;
    public Node prev;

    public Node(Media data){
        this.data = data;
        this.nestedNode = null;
        this.next = null;
    }
}
```

```

        this.prev = null;
    }

    public Object getData() {
        return this.data;
    }
}

```

```

package Nodes;
import Data.Media;

public class MediaNode extends Node {
    public MediaNode(Media data){
        super(data);
    }

    public Media getMedia() {
        return (Media) getData();
    }

    public void displayInfo() {
        Media media = getMedia();
        System.out.println("Media: " + media.toString());
    }
}

```

The class is inheritance from Node, and this class has 2 another method, which getMedia for retrieving the data from current Node and display info

```

package List;
import Data.*;
import Nodes.*;

public abstract class LinkedList {
    public Node head;
    public Node tail;
    public int size;

    public LinkedList(){
        this.head = null;
        this.tail = null;
        this.size = 0;
    }

    public abstract void add(Media media);
    public abstract boolean delete(String title);
    public abstract Media search(String title);
    public abstract void display();
}

```

```
public int getSize() { return size; }  
}
```

The LinkedList class is an **abstract class** that serves as a blueprint for different types of linked lists (SinglyLinkedList and DoublyLinkedList). It contains three main properties: head (first node), tail (last node), and size (number of elements). The constructor initializes an empty list with all properties set to null/0.

This class defines four abstract methods (add, delete, search, display) that **must be implemented** by child classes. It's abstract because these methods work differently between SinglyLinkedList (only has next pointer) and DoublyLinkedList (has both next and previous pointers). This design ensures code reusability and consistency while allowing flexible implementations based on each list type's characteristics.

LINKEDLISTS

```
package List;  
import Data.*;  
import Nodes.*;  
  
public class PodcastManager extends LinkedList {  
    @Override  
    public void add(Media media) {  
        MediaNode newNode = new MediaNode(media);  
  
        if (head == null) {  
            head = tail = newNode;  
        } else {  
            tail.next = newNode;  
            newNode.prev = tail;  
            tail = newNode;  
        }  
        size++;  
    }  
  
    @Override  
    public boolean delete(String title) {  
        if (head == null) return false;  
  
        Node current = head;  
        while (current != null) {  
            if (current.data.title.equalsIgnoreCase(title)) {  
                if (current == head) {  
                    head = head.next;  
                }  
                if (current == tail) {  
                    tail = tail.prev;  
                }  
                current.prev.next = current.next;  
                current.next.prev = current.prev;  
                return true;  
            }  
            current = current.next;  
        }  
        return false;  
    }  
  
    @Override  
    public boolean search(String title) {  
        if (head == null) return false;  
  
        Node current = head;  
        while (current != null) {  
            if (current.data.title.equalsIgnoreCase(title)) {  
                return true;  
            }  
            current = current.next;  
        }  
        return false;  
    }  
  
    @Override  
    public void display() {  
        if (head == null) {  
            System.out.println("List is empty");  
            return;  
        }  
        Node current = head;  
        while (current != null) {  
            System.out.print(current.data.title + " ");  
            current = current.next;  
        }  
        System.out.println();  
    }  
}
```

```

        if (head != null) {
            head.prev = null;
        } else {
            tail = null;
        }
    }
    else if (current == tail) {
        tail = tail.prev;
        tail.next = null;
    }
    else {
        current.prev.next = current.next;
        current.next.prev = current.prev;
    }

    size--;
    return true;
}
current = current.next;
}

return false;
}

@Override
public Media search(String title) {
    Node current = head;
    while (current != null) {
        if (current.data != null &&
current.data.title.equalsIgnoreCase(title)) {
            return current.data;
        }
        current = current.next;
    }
    return null;
}

@Override
public void display() {
    if (head == null) {
        System.out.println(" (No podcasts available)");
        return;
    }

    Node current = head;
    int index = 1;
    while (current != null) {
        System.out.print(" " + index + ". ");
    }
}

```

```

        current.data.display();
        current = current.next;
        index++;
    }
}
}

```

The PodcastManager class extends LinkedList and implements a **doubly linked list** for managing podcast media. It overrides all four abstract methods from the parent class with specific implementations for doubly linked list operations.

```

package List;
import Data.*;
import Nodes.*;

public class SongManager extends LinkedList {
    @Override
    public void add(Media media){
        MediaNode newNode = new MediaNode(media);

        if(head == null){
            head = tail = newNode;
        } else {
            tail.next = newNode;
            tail = newNode;
        }
        size++;
    }

    @Override
    public boolean delete(String title) {
        if (head == null) return false;

        if (head.data.title.equalsIgnoreCase(title)) {
            head = head.next;
            if (head == null) {
                tail = null;
            }
            size--;
            return true;
        }

        Node current = head;
        while (current.next != null) {
            if (current.next.data.title.equalsIgnoreCase(title)) {
                if (current.next == tail) {
                    tail = current;
                }
                current.next = current.next.next;
            }
        }
    }
}

```

```

        size--;
        return true;
    }
    current = current.next;
}
return false;
}

@Override
public Media search(String title) {
    Node current = head;
    while (current != null) {
        if (current.data != null &&
current.data.title.equalsIgnoreCase(title)) {
            return current.data;
        }
        current = current.next;
    }
    return null;
}

@Override
public void display() {
    if (head == null) {
        System.out.println(" (No songs available)");
        return;
    }

    Node current = head;
    int index = 1;
    while (current != null) {
        System.out.print(" " + index + ". ");
        current.data.display();
        current = current.next;
        index++;
    }
}
}

```

The SongManager class extends LinkedList and implements a **singly linked list** for managing song media. It overrides all four abstract methods from the parent class with specific implementations for singly linked list operations.

The add method creates a new MediaNode and appends it to the end of the list, only maintaining the next pointer (no prev pointer unlike doubly linked list). The delete method searches for a song by title and removes it by updating only the next pointer of the previous node, handling two cases: deleting the head or any other node. The search method traverses the

list forward to find and return a media object matching the given title. The display method prints all songs in a numbered list format, or shows a message if the list is empty. All operations only use forward traversal since singly linked lists only have next pointers, making it simpler but less flexible than doubly linked lists.

```
package List;
import Data.*;
import Nodes.*;

public class VideoManager extends LinkedList {

    @Override
    public void add(Media media) {
        MediaNode newNode = new MediaNode(media);

        if (head == null) {
            head = tail = newNode;
            newNode.next = head;
        } else {
            tail.next = newNode;
            newNode.next = head;
            tail = newNode;
        }
        size++;
    }

    @Override
    public boolean delete(String title) {
        if (head == null) return false;

        if (head == tail && head.data.title.equalsIgnoreCase(title)) {
            head = tail = null;
            size--;
            return true;
        }

        if (head.data.title.equalsIgnoreCase(title)) {
            head = head.next;
            tail.next = head;
            size--;
            return true;
        }

        Node current = head;
        do {
            if (current.next.data.title.equalsIgnoreCase(title)) {
                if (current.next == tail) {
                    tail = current;
                }
            }
        } while (current.next != null);
    }
}
```



```

        }
        current.next = current.next.next;
        size--;
        return true;
    }
    current = current.next;
} while (current != head);

return false;
}

@Override
public Media search(String title) {
    if (head == null) return null;

    Node current = head;
    do {
        if (current.data != null &&
current.data.title.equalsIgnoreCase(title)) {
            return current.data;
        }
        current = current.next;
    } while (current != head);

    return null;
}

@Override
public void display() {
    if (head == null) {
        System.out.println(" (No videos available)");
        return;
    }

    Node current = head;
    int index = 1;
    do {
        System.out.print(" " + index + ". ");
        current.data.display();
        current = current.next;
        index++;
    } while (current != head);
}

public void displayCircular(int loops) {
    if (head == null) {
        System.out.println(" (No videos available)");
        return;
    }

```

```

    }

    System.out.println("\n=== Video Playlist (Circular - " + loops + "
loops) ===");
    Node current = head;
    int count = 0;
    int maxDisplay = size * loops;

    do {
        System.out.print("  " + (count + 1) + ". ");
        current.data.display();
        current = current.next;
        count++;
    } while (count < maxDisplay);
}
}

```

The VideoManager class extends LinkedList and implements a **circular singly linked list** for managing video media. It overrides all four abstract methods from the parent class with specific implementations for circular linked list operations, where the last node points back to the first node. The other method is similar with the other two classes

```

package List;
import Data.*;

public class Manager {
    SongManager songManager;
    PodcastManager podcastManager;
    VideoManager videoManager;

    public Manager(){
        this.songManager = new SongManager();
        this.podcastManager = new PodcastManager();
        this.videoManager = new VideoManager();
    }

    //Song
    public void addSong(String title, String artist, int duration, String
album, String genre) {
        Song song = new Song(title, artist, duration, album, genre);
        songManager.add(song);
        System.out.println("✓ Song added: " + title);
    }

    public boolean deleteSong(String title) {
        boolean deleted = songManager.delete(title);
        if (deleted) {
            System.out.println("✓ Song deleted: " + title);
        } else {

```

```

        System.out.println("X Song not found: " + title);
    }
    return deleted;
}

public Media searchSong(String title) {
    return songManager.search(title);
}

public void displayAllSongs() {
    System.out.println("\n=== All Songs ===");
    songManager.display();
    System.out.println("Total: " + songManager.getSize() + " songs");
}

//Podcast
public void addPodcast(String title, String artist, int duration, String
host, String category, int episode) {
    Podcast podcast = new Podcast(title, artist, duration, host,
category, episode);
    podcastManager.add(podcast);
    System.out.println("✓ Podcast added: " + title);
}

public boolean deletePodcast(String title) {
    boolean deleted = podcastManager.delete(title);
    if (deleted) {
        System.out.println("✓ Podcast deleted: " + title);
    } else {
        System.out.println("X Podcast not found: " + title);
    }
    return deleted;
}

public Media searchPodcast(String title) {
    return podcastManager.search(title);
}

public void displayAllPodcasts() {
    System.out.println("\n=== All Podcasts ===");
    podcastManager.display();
    System.out.println("Total: " + podcastManager.getSize() + "
podcasts");
}

//Video
public void addVideo(String title, String channel, int duration, String
resolution) {

```

```

        Video video = new Video(title, channel, duration, resolution);
        videoManager.add(video);
        System.out.println("✓ Video added: " + title);
    }

    public boolean deleteVideo(String title) {
        boolean deleted = videoManager.delete(title);
        if (deleted) {
            System.out.println("✓ Video deleted: " + title);
        } else {
            System.out.println("X Video not found: " + title);
        }
        return deleted;
    }

    public Media searchVideo(String title) {
        return videoManager.search(title);
    }

    public void displayAllVideos() {
        System.out.println("\n=== All Videos ===");
        videoManager.display();
        System.out.println("Total: " + videoManager.getSize() + " videos");
    }

    public void displayVideoPlaylist(int loops) {
        videoManager.displayCircular(loops);
    }

    public void displayStatistics() {
        System.out.println("\n┌───────────────────────────────────┐");
        System.out.println("│          SPOTIFY SYSTEM STATS          │");
        System.out.println("└───────────────────────────────────┘");
        System.out.println("│ Total Songs:      " + String.format("%12d",
songManager.getSize()) + " │");
        System.out.println("│ Total Podcasts:   " + String.format("%12d",
podcastManager.getSize()) + " │");
        System.out.println("│ Total Videos:    " + String.format("%12d",
videoManager.getSize()) + " │");
        System.out.println("│ ──────────────────── │");
        System.out.println("│ Total Media:      " + String.format("%12d",
songManager.getSize() + podcastManager.getSize() +
videoManager.getSize()) + " │");
        System.out.println("└───────────────────────────────────┘");
    }
}

```

The Manager class serves as a **facade/controller** that manages three different types of media collections: songs (singly linked list), podcasts (doubly linked list), and videos (circular

linked list). It encapsulates three manager objects (SongManager, PodcastManager, VideoManager) and provides a unified interface for all media operations.

The class implements CRUD operations (Create, Read, Update, Delete) for each media type with corresponding methods like addSong, deleteSong, searchSong, and displayAllSongs, with similar patterns for podcasts and videos. Each method provides user feedback through console messages (✓ for success, X for failure). The displayStatistics method presents a formatted overview of all media counts across the three collections. This design pattern simplifies the main application code by centralizing all media management logic in one place, allowing the main program to interact with a single Manager object instead of managing three separate list managers directly.

DATA (MEDIA CLASS)

```
package Data;

public abstract class Media {
    public String title;
    public String artist;
    public int duration;

    public Media(String title, String artist, int duration){
        this.title = title;
        this.artist = artist;
        this.duration = duration;
    }

    public abstract String getType();
    public abstract void display();

    public String getFormattedDuration(){
        int minutes = duration / 60;
        int seconds = duration % 60;
        return minutes + ":" + (seconds < 10 ? "0" + seconds : seconds);
    }
}
```

The Media class is an **abstract base class** that serves as a blueprint for different types of media content (Song, Podcast, Video). It contains three common properties shared by all media types: title, artist (or creator), and duration in seconds. The constructor initializes these three fundamental attributes that every media object must have.

The class defines two abstract methods that **must be implemented** by child classes: getType() to return the media type as a string, and display() to show media-specific information in a formatted way. It also provides a concrete utility method getFormattedDuration() that

converts the duration from seconds into a readable "minutes:seconds" format (e.g., 185 seconds becomes "3:05").

```
package Data;

public class Podcast extends Media {
    public String host;
    public String category;
    public int episode;

    public Podcast(String title, String artist, int duration, String host,
String category, int episode){
        super(title, artist, duration);
        this.host = host;
        this.category = category;
        this.episode = episode;
    }

    @Override
    public String getType() { return "Podcast"; }

    @Override
    public void display() {
        System.out.println("[PODCAST] Episode " + episode + ": " + title +
            " | Host: " + host + " | Category: " + category +
            " | Duration: " + getFormattedDuration());
    }
}
```

The Podcast class extends Media and represents podcast episodes with three additional specific attributes: host (the podcast host's name), category (the genre or topic), and episode (the episode number). The constructor calls super() to initialize the inherited properties (title, artist, duration) from the parent Media class, then sets the podcast-specific attributes.

```
package Data;

public class Song extends Media{
    public String album;
    public String genre;

    public Song(String title, String artist, int duration, String album,
String genre){
        super(title, artist, duration);
        this.album = album;
        this.genre = genre;
    }

    @Override
```

```

        public String getType(){ return "Song"; }

        @Override
        public void display(){ System.out.println("[SONG] " + title + " - " +
artist + " | Album: " + album +
                        " | Genre: " + genre + " | Duration: " +
getFormattedDuration());}
    }

```

The Song class extends Media and represents music tracks with two additional specific attributes: album (the album name the song belongs to) and genre (the music genre like Pop, Rock, Jazz, etc.). The constructor calls super() to initialize the inherited properties (title, artist, duration) from the parent Media class, then sets the song-specific attributes.

```

package Data;

public class Video extends Media {
    public String channel;
    public String resolution;
    public long views;

    public Video(String title, String channel, int duration, String
resolution) {
        super(title, channel, duration);
        this.channel = channel;
        this.resolution = resolution;
        this.views = 0;
    }

    @Override
    public String getType() {
        return "Video";
    }

    @Override
    public void display() {
        System.out.println("[VIDEO] " + title + " | Channel: " + channel +
                        " | Resolution: " + resolution + " | Duration: " +
getFormattedDuration() +
                        " | Views: " + views);
    }

    public void watch() {
        views++;
        System.out.println("Watching: " + title + " (Views: " + views +
        ")");
    }
}

```

```
}
```

The Video class extends Media and represents video content with three specific attributes: channel (the creator's channel name), resolution (video quality like "1080p" or "4K"), and views (number of times the video has been watched, initialized to 0). The constructor calls super() to initialize the inherited properties, passing channel as the artist parameter to the parent class, then sets the video-specific attributes.

MAIN CLASS

```
import Data.Media;
import List.Manager;

public class Spotify {
    public static void main(String[] args) {
        Manager manage = new Manager();

        manage.addSong("Kimi no Namae", "Chiai Fujiwika", 192, "Anime", "J-
Pop");
        manage.addSong("Shinunoga E-Wa", "Fujii Kaze", 215, "Love", "J-
Pop");

        manage.addPodcast("Tech Talk Daily", "Jane Doe", 1500, "Technology",
"Tech World", 45);
        manage.addPodcast("History Hour", "John Smith", 3600, "History",
"Past Times", 12);

        manage.addVideo("Java Tutorial for Beginners", "Programming Hub",
3600, "1080p");
        manage.addVideo("Nature Documentary", "National Geographic", 2700,
"4K");
        manage.addVideo("Cooking Masterclass", "Gordon Ramsay", 1800,
"1080p");

        System.out.println("\n\n>>> Displaying All Media...");
        manage.displayAllSongs();
        manage.displayAllPodcasts();
        manage.displayAllVideos();

        System.out.println("\n\n>>> Search Operations...");
        searchSong(manage, "Shinunoga E-Wa");
        searchPodcast(manage, "History Hour");
        searchVideo(manage, "Java Tutorial for Beginners");

        System.out.println("\n\n>>> Search Operations Completed.");

        System.out.println("\n>>> Delete Operations...");
```



```

        manage.deleteSong("Kimi no Namae");
        manage.deletePodcast("Tech Talk Daily");
        manage.deleteVideo("Cooking Masterclass");

        System.out.println("\n\n>>> Displaying All Media After
Deletions...");
        manage.displayAllSongs();
        manage.displayAllPodcasts();
        manage.displayAllVideos();

        manage.displayStatistics();
    }

    public static void searchSong(Manager manage, String title){
        Media found = manage.searchSong(title);
        if (found != null) {
            System.out.print("Found: ");
            found.display();
        }
    }

    public static void searchPodcast(Manager manage, String title){
        Media found = manage.searchPodcast(title);
        if (found != null) {
            System.out.print("Found: ");
            found.display();
        }
    }

    public static void searchVideo(Manager manage, String title){
        Media found = manage.searchVideo(title);
        if (found != null) {
            System.out.print("Found: ");
            found.display();
        }
    }
}

```

The Spotify class contains the main method and serves as the **entry point** of the application, demonstrating the complete functionality of the media management system. It creates a Manager object to handle all operations and uses helper methods (searchSong, searchPodcast, searchVideo) to perform search operations with formatted output.

C. RESULT

```
✓ Song added: Kimi no Namae
✓ Song added: Shinunoga E-Wa
✓ Podcast added: Tech Talk Daily
✓ Podcast added: History Hour
✓ Video added: Java Tutorial for Beginners
✓ Video added: Nature Documentary
✓ Video added: Cooking Masterclass

>>> Displaying All Media...

=== All Songs ===
  1. [SONG] Kimi no Namae - Chial Fujiwika | Album: Anime | Genre: J-Pop | Duration: 3:12
  2. [SONG] Shinunoga E-Wa - Fujii Kaze | Album: Love | Genre: J-Pop | Duration: 3:35
Total: 2 songs

=== All Podcasts ===
  1. [PODCAST] Episode 45: Tech Talk Daily | Host: Technology | Category: Tech World | Duration: 25:00
  2. [PODCAST] Episode 12: History Hour | Host: History | Category: Past Times | Duration: 60:00
Total: 2 podcasts

=== All Videos ===
  1. [VIDEO] Java Tutorial for Beginners | Channel: Programming Hub | Resolution: 1080p | Duration: 60:00 | Views: 0
  2. [VIDEO] Nature Documentary | Channel: National Geographic | Resolution: 4K | Duration: 45:00 | Views: 0
  3. [VIDEO] Cooking Masterclass | Channel: Gordon Ramsay | Resolution: 1080p | Duration: 30:00 | Views: 0
Total: 3 videos
```

```
>>> Search Operations...
Found: [SONG] Shinunoga E-Wa - Fujii Kaze | Album: Love | Genre: J-Pop | Duration: 3:35
Found: [PODCAST] Episode 12: History Hour | Host: History | Category: Past Times | Duration: 60:00
Found: [VIDEO] Java Tutorial for Beginners | Channel: Programming Hub | Resolution: 1080p | Duration: 60:00 | Views: 0

>>> Search Operations Completed.

>>> Delete Operations...
✓ Song deleted: Kimi no Namae
✓ Podcast deleted: Tech Talk Daily
✓ Video deleted: Cooking Masterclass

>>> Displaying All Media After Deletions...

=== All Songs ===
  1. [SONG] Shinunoga E-Wa - Fujii Kaze | Album: Love | Genre: J-Pop | Duration: 3:35
Total: 1 songs

=== All Podcasts ===
  1. [PODCAST] Episode 12: History Hour | Host: History | Category: Past Times | Duration: 60:00
Total: 1 podcasts

=== All Videos ===
  1. [VIDEO] Java Tutorial for Beginners | Channel: Programming Hub | Resolution: 1080p | Duration: 60:00 | Views: 0
  2. [VIDEO] Nature Documentary | Channel: National Geographic | Resolution: 4K | Duration: 45:00 | Views: 0
Total: 2 videos
```

SPOTIFY SYSTEM STATS	
Total Songs:	1
Total Podcasts:	1
Total Videos:	2
Total Media:	4

Overall, this program demonstrates the practical implementation of all major variations of linked lists namely singly linked lists, doubly linked lists, and circular linked lists within the context of a Spotify system. Each type of list is applied to a specific use case the singly linked. Across these structures, the program makes use of core linked list algorithms such as insertion at the beginning, end, deletion of nodes at various positions; searching for elements; traversing and displaying nodes; and clearing entire lists. This not only showcases how fundamental linked list operations can be implemented in Java, but also illustrates how data structures can be mapped to real-world applications like music management systems.